Deep Neural Network (CNN - Convolutional Neural Network)

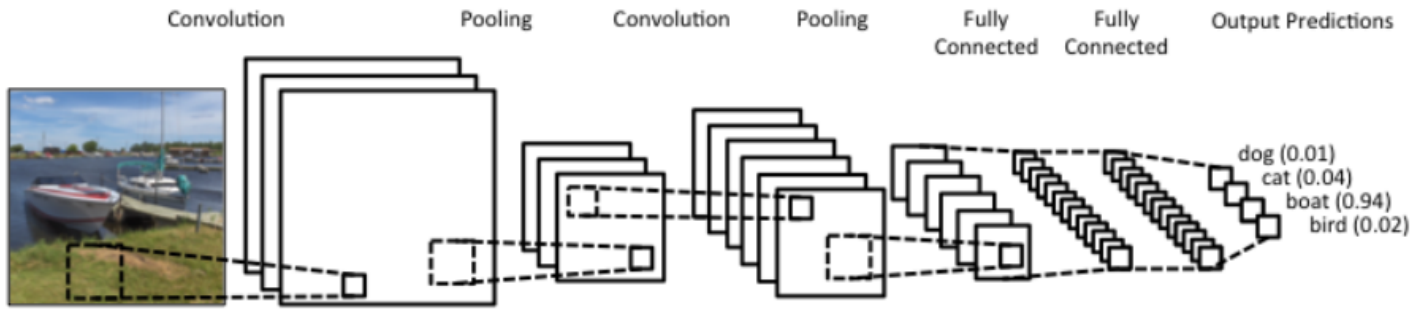
Convolutional Neural Network Structure

Convolutional Neural Network Functions

Convolutional Neural Network Training

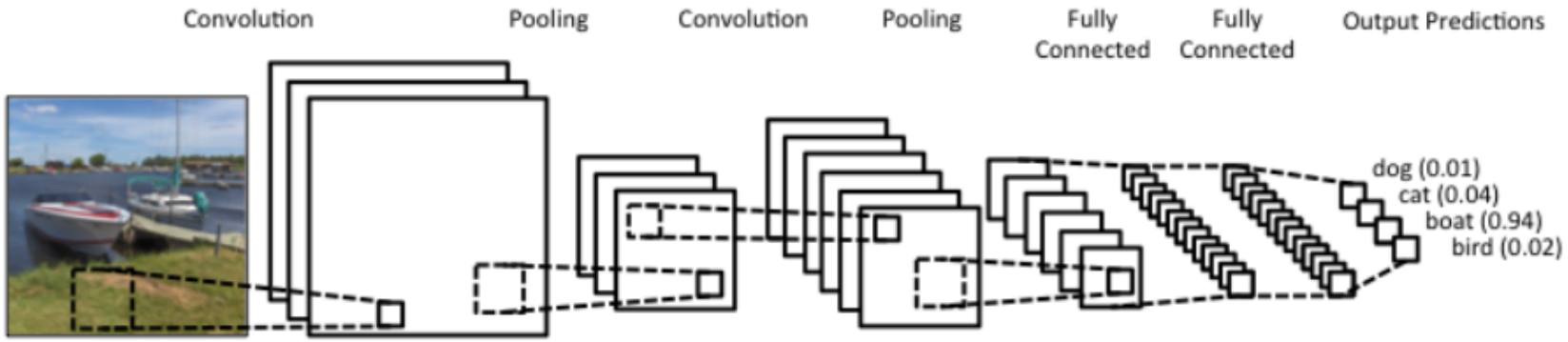
Contents

- CNN Architecture
- CNN Capability
- CNN Learning



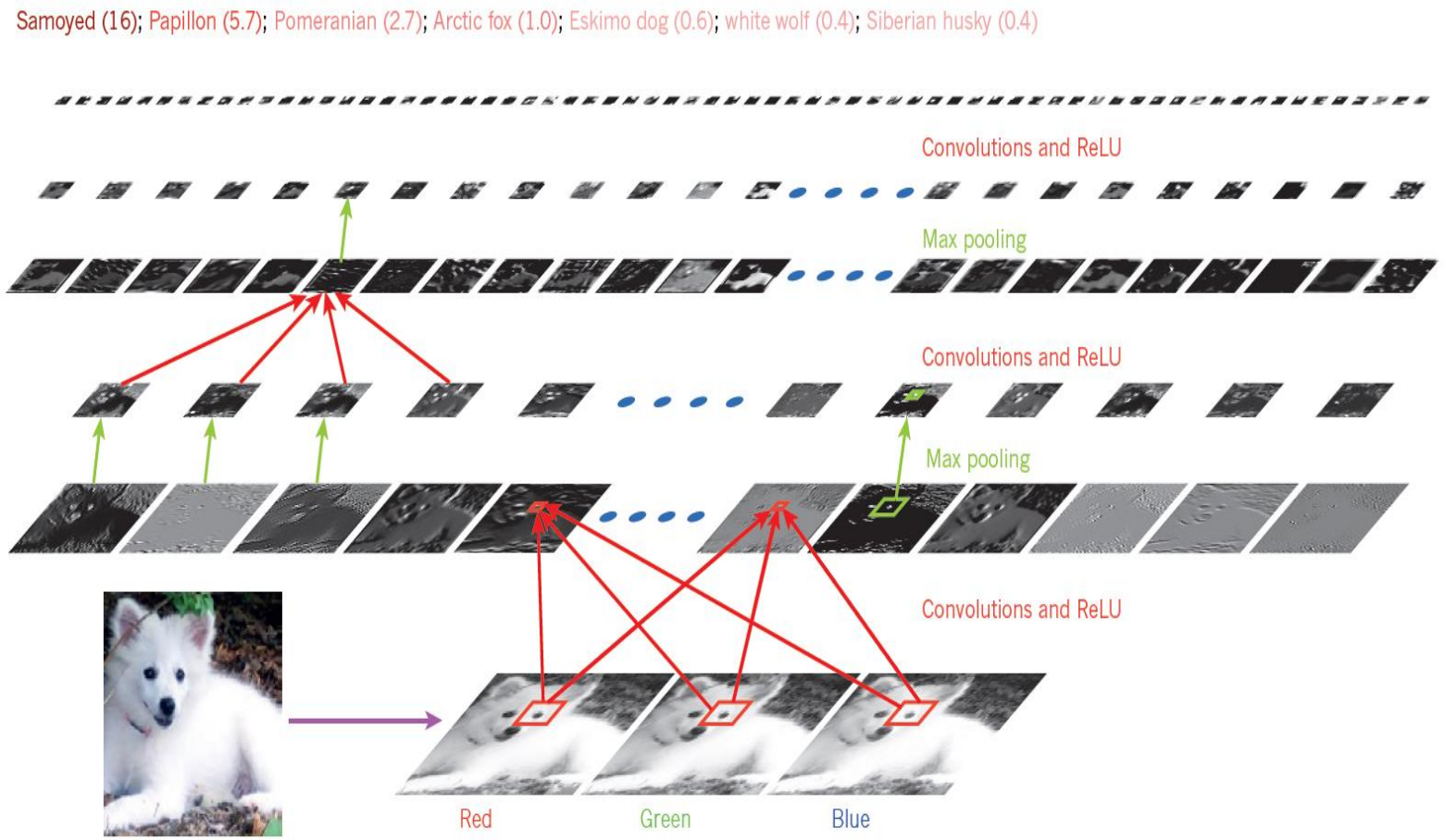
CNN - Architecture

- **Deep feed-forward network**
  - **Convolution layers**
  - **Pooling layers**
  - **Fully-Connected layers**



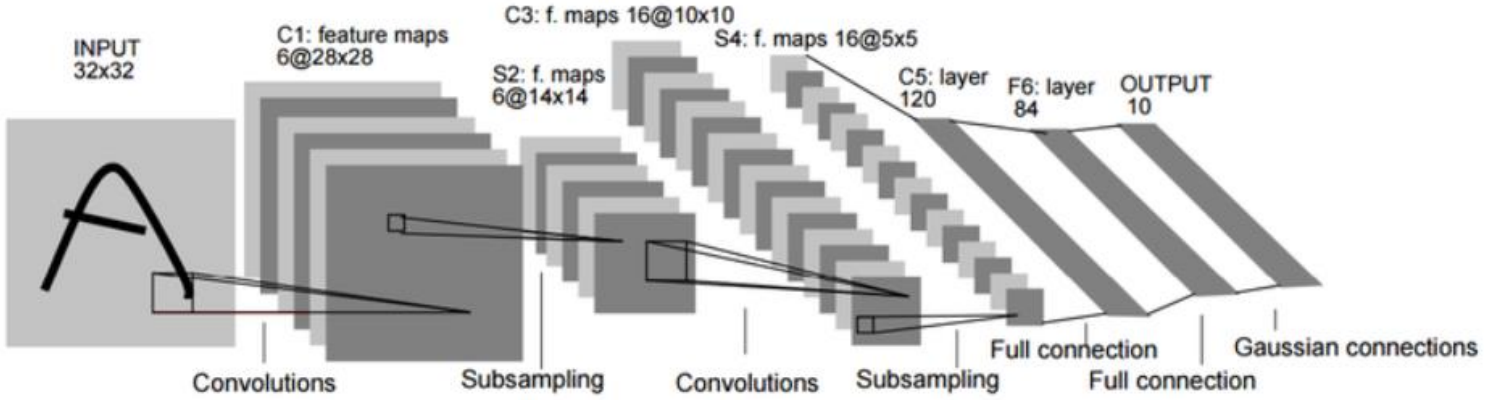
CNN - Architecture

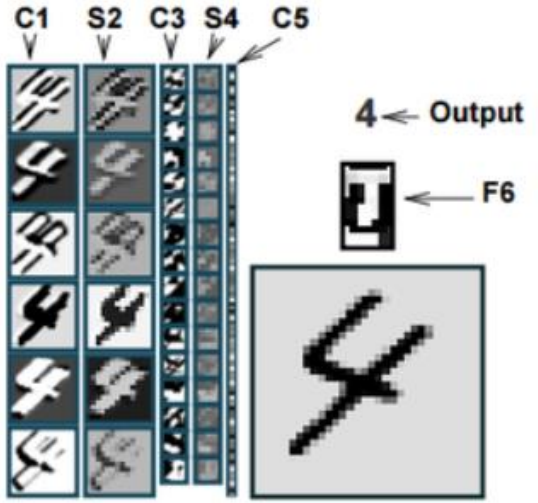
- **Image Classification with CNN**



CNN - Architecture

- **Character Recognition with CNN**

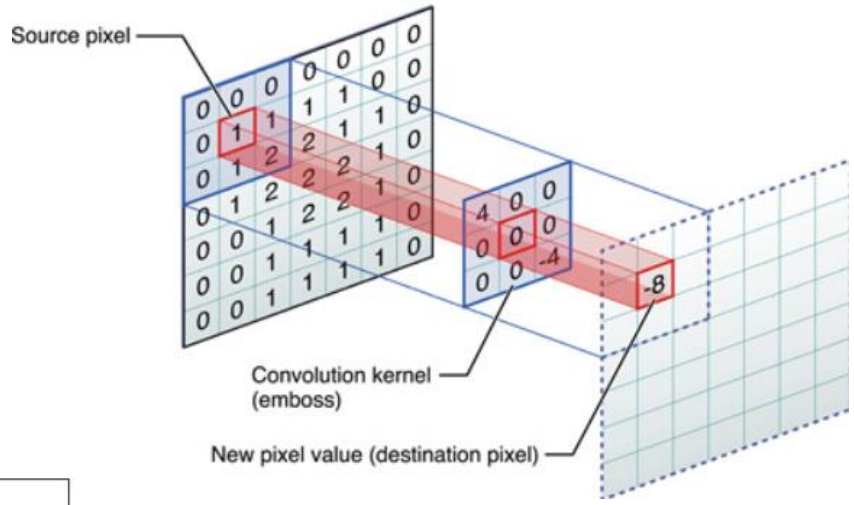


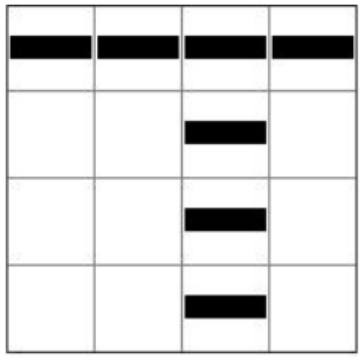


CNN - Architecture

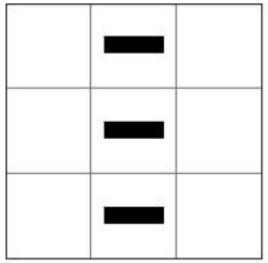
- **Image Convolution**

$$f(x) * g(x) = \int_{t=-\infty}^{\infty} f(t)g(x-t)dx$$





$f(x)$



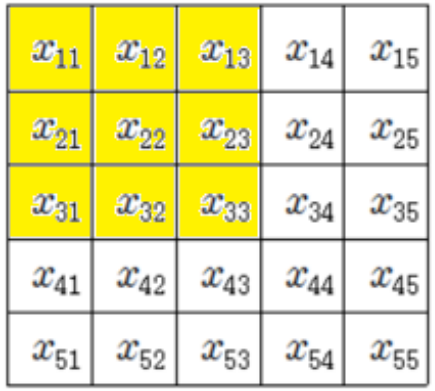
$g(x)$

1	3
0	3

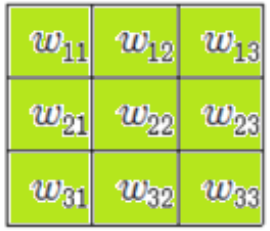
$f(x)$
$* g(x)$

Convolution

- Covolution Operation



Input



Filter (Kernel, Mask)



Convolution Result

$$y_{11} = w_{11}x_{11} + w_{12}x_{12} + w_{13}x_{13} + w_{21}x_{21} + w_{22}x_{22} + w_{23}x_{23} + w_{31}x_{31} + w_{32}x_{32} + w_{33}x_{33} + w_0$$

Convolution

- Convolution Example



Input

1	0	1
0	1	0
1	0	1

Filter

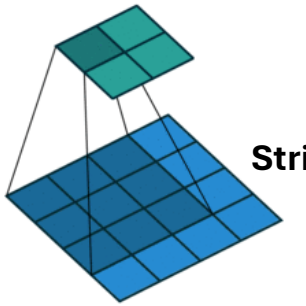
4	3	4
2	4	3
2	3	4

Output

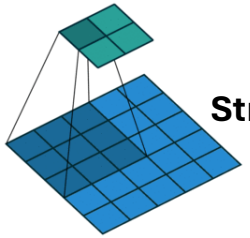
$$y_{11} = w_{11}x_{11} + w_{12}x_{12} + w_{13}x_{13} + w_{21}x_{21} + w_{22}x_{22} + w_{23}x_{23} + w_{31}x_{31} + w_{32}x_{32} + w_{33}x_{33} + w_0$$

Convolution

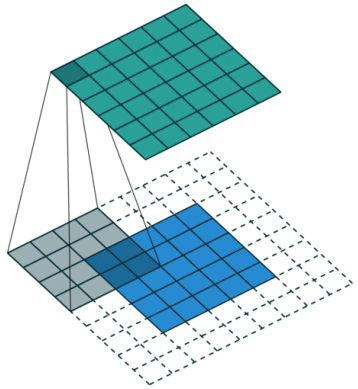
- **S**tride
  - Interval of Kernel movement
- **P**adding
  - Number of 0-valued rows(or columns) along the image boundary



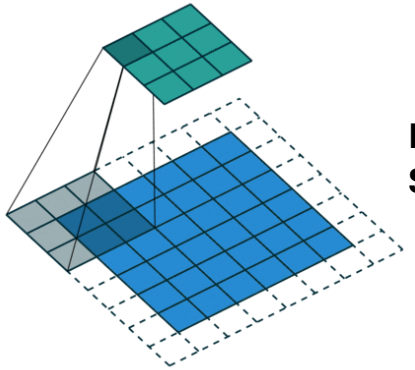
Stride: 1



Stride : 2



Padding : 2
Stride : 1



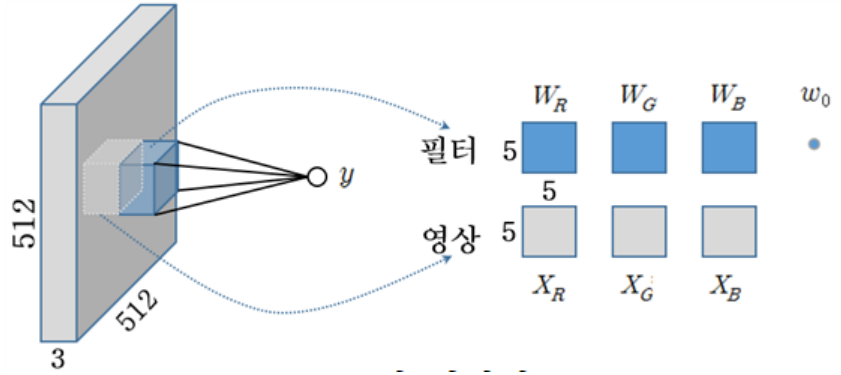
Padding: 1
Stride : 2

Convolution

- **Convolution for Color Image**
  - **Multi-dimensional matrix**



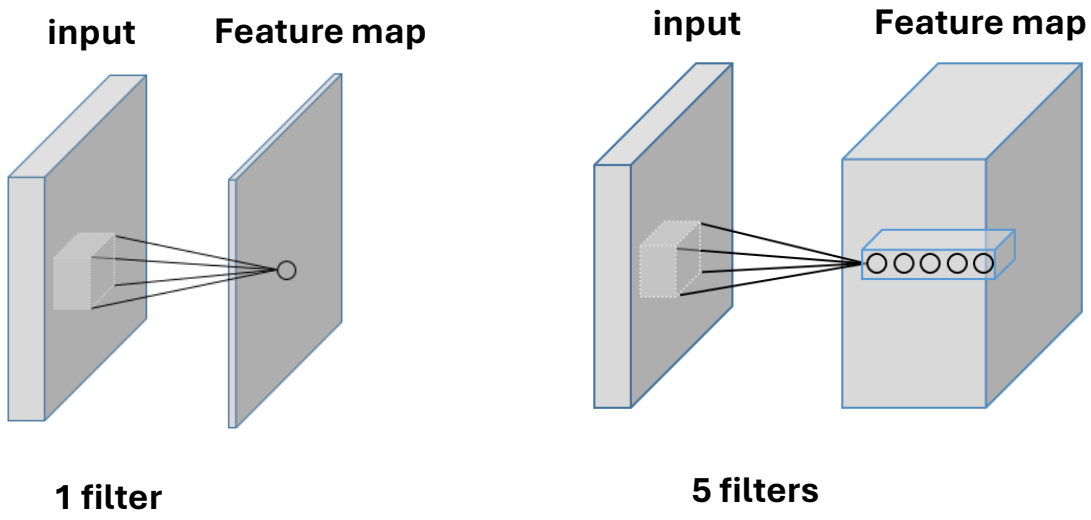
  - **Convolution on a Color Image**



$$y = X_R * W_R + X_G * W_G + X_B * W_B + w_0$$

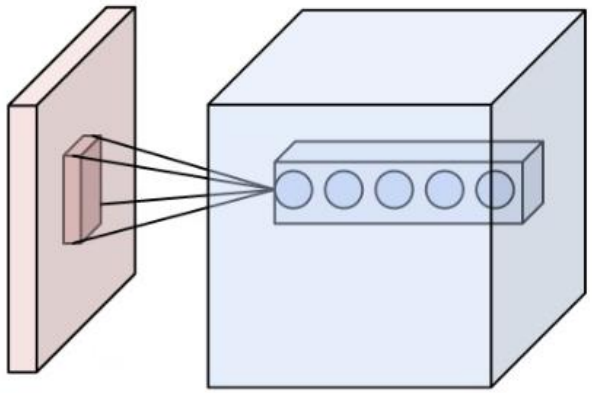
Convolution

- Feature Map
  - A 2-D matrix is obtained by applying a convolution filter
  - k filters → feature map of depth k



CNN - Architecture

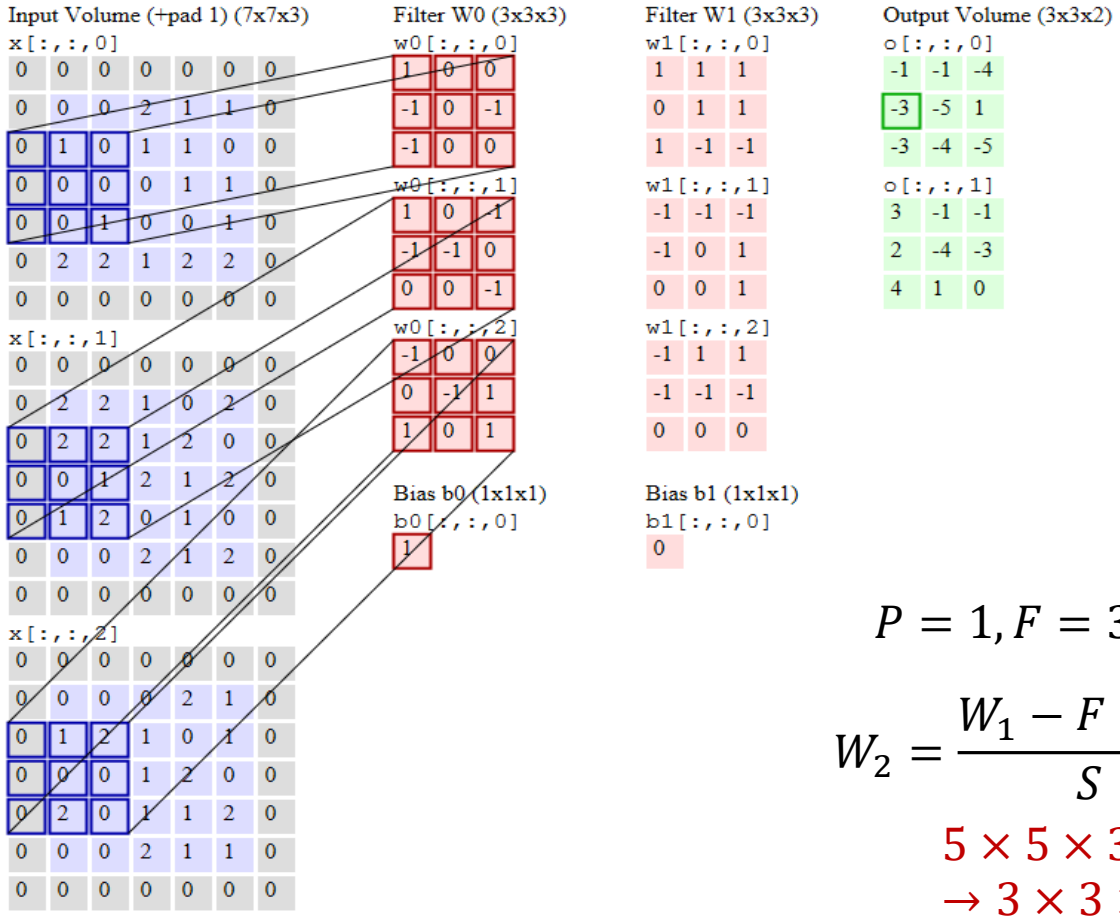
- **Convolution layer**



- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$
  - $D_2 = K$

CNN - Architecture

- **Convolution layer - example**



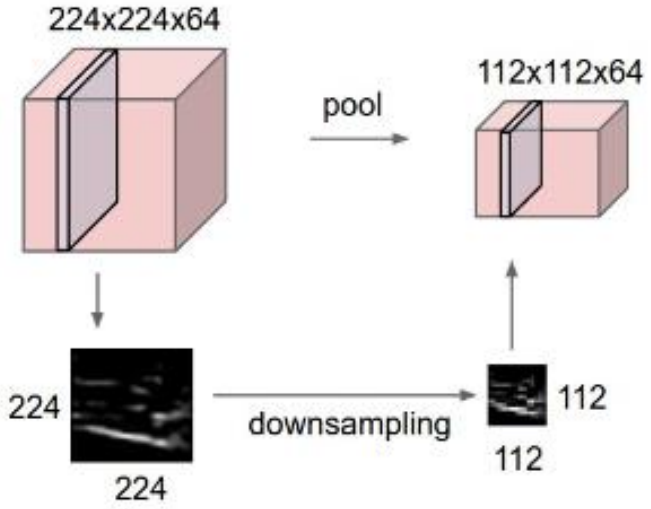
$$P = 1, F = 3, S = 2, K = 2$$

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$5 \times 5 \times 3$
$\rightarrow 3 \times 3 \times 2$

CNN - Architecture

- **Pooling layer**



- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires three hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$

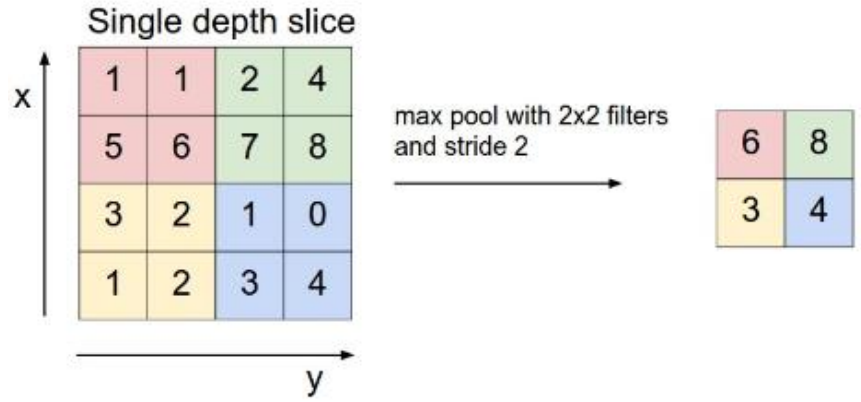
$F = 2, S = 2$

$$W_2 = \frac{W_1 - F}{S} + 1$$

$4 \times 4 \times 1$

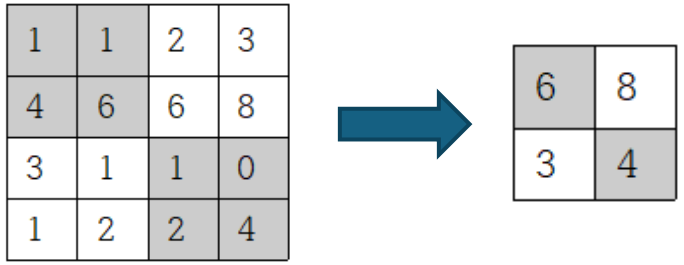
$\rightarrow 2 \times 2 \times 1$

max pooling, or mean pooling

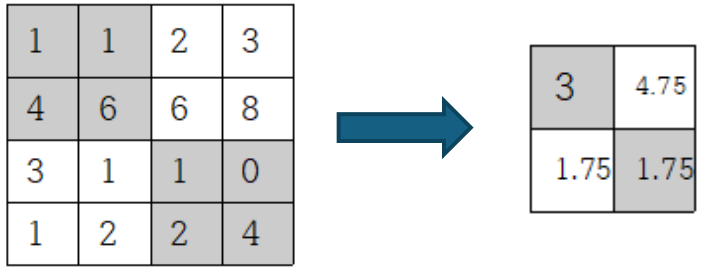


Pooling

- Max pooling



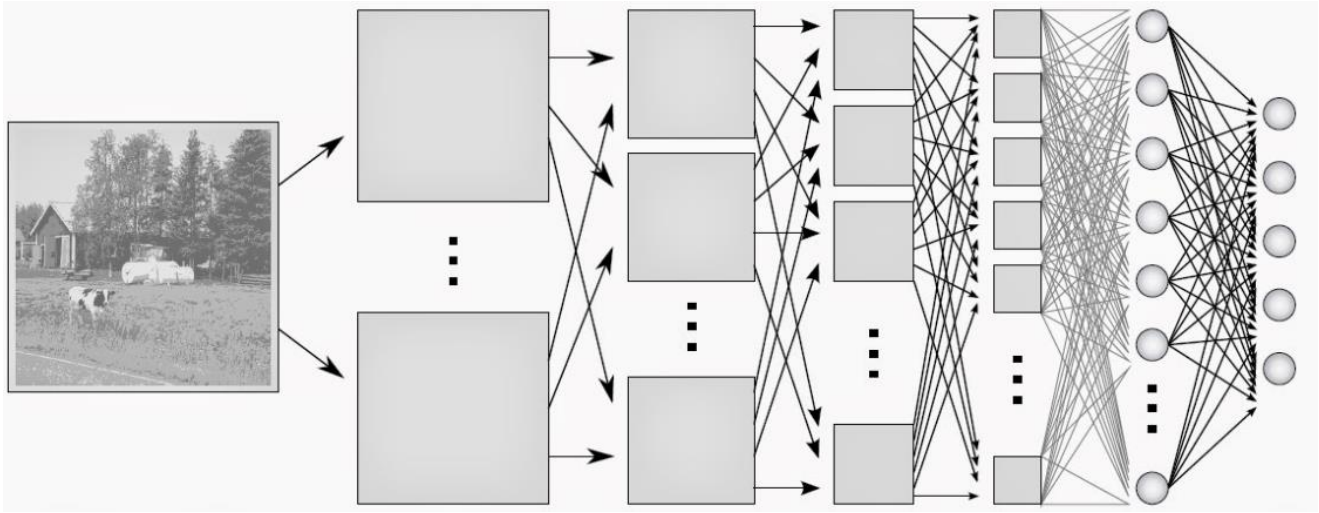
- Mean pooling



- Stochastic pooling

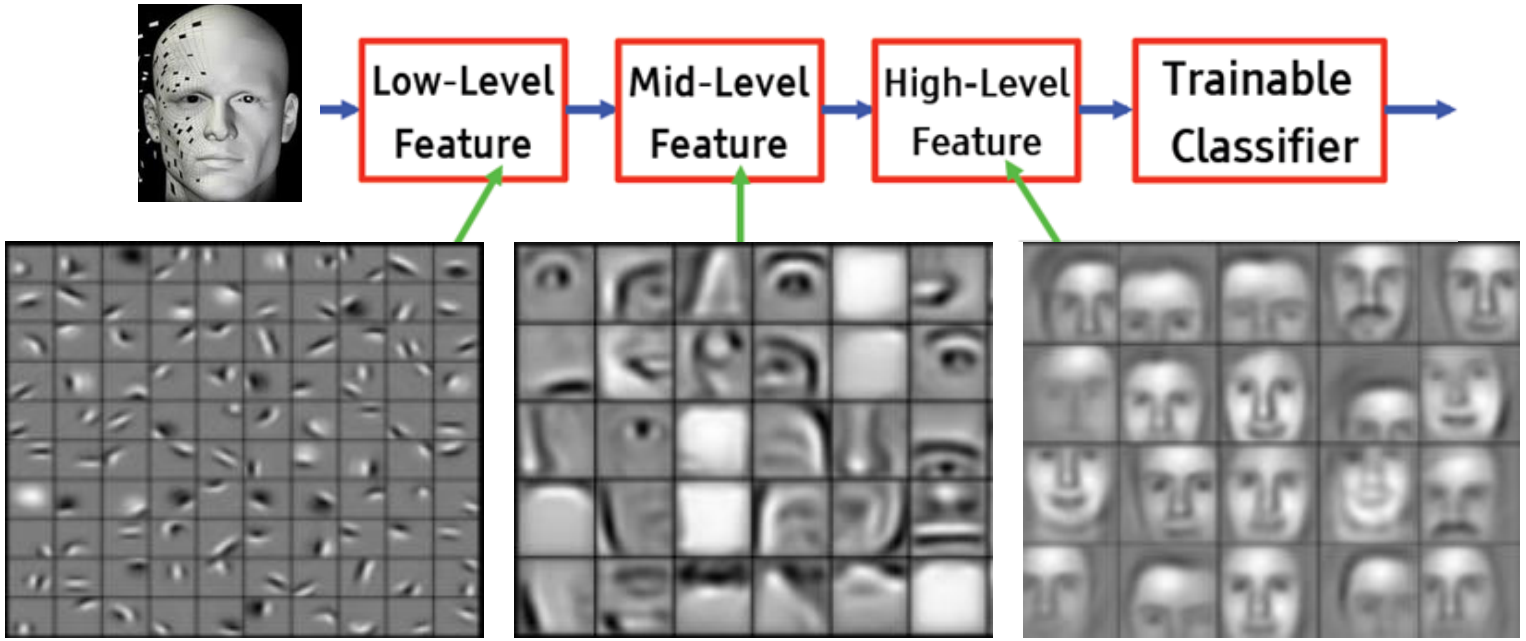
CNN - Architecture

- **FC layer: MLP layer**



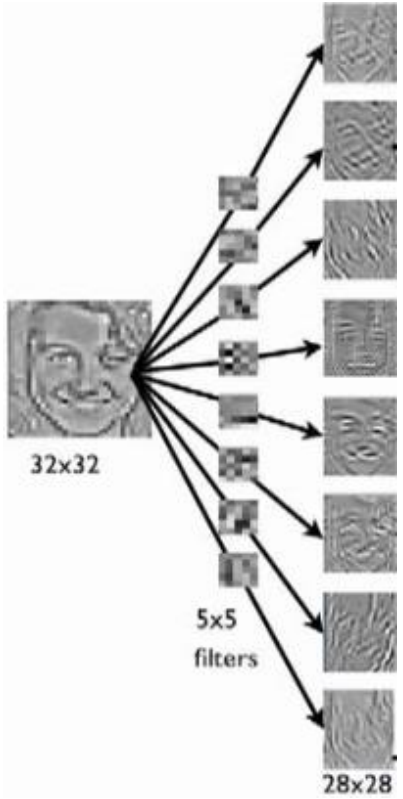
CNN - Architecture

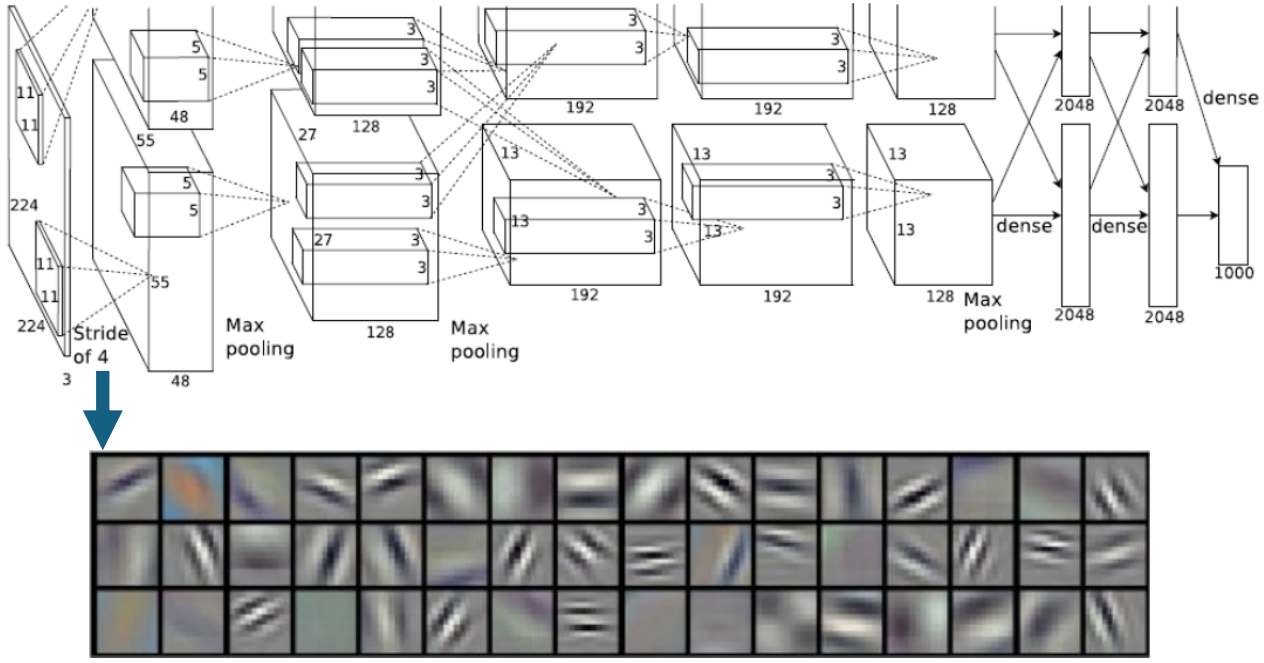
- **Feature Maps - multiple levels**



CNN - Architecture

- **Filters(Kernels)**

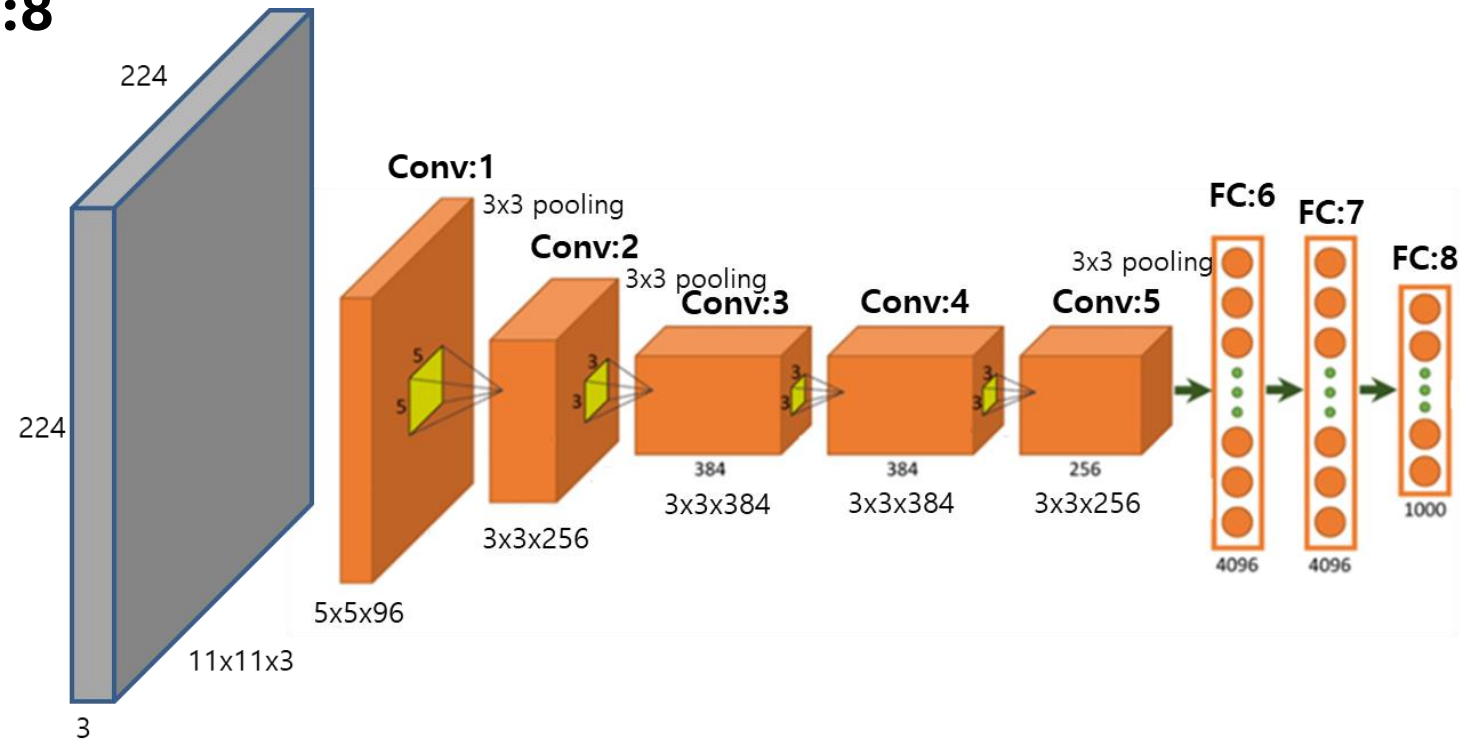




48 filters learned by the first conv. layer in [Krizenhvsky 2012]

CNN Architecture

- **Example CNN (AlexNet, 2012)**
  - **Conv:1-Pool:1-Conv:2-Pool:2-Conv:3-Conv:4-Conv:5-Pool:4-FC:6-FC:7-FC:8**



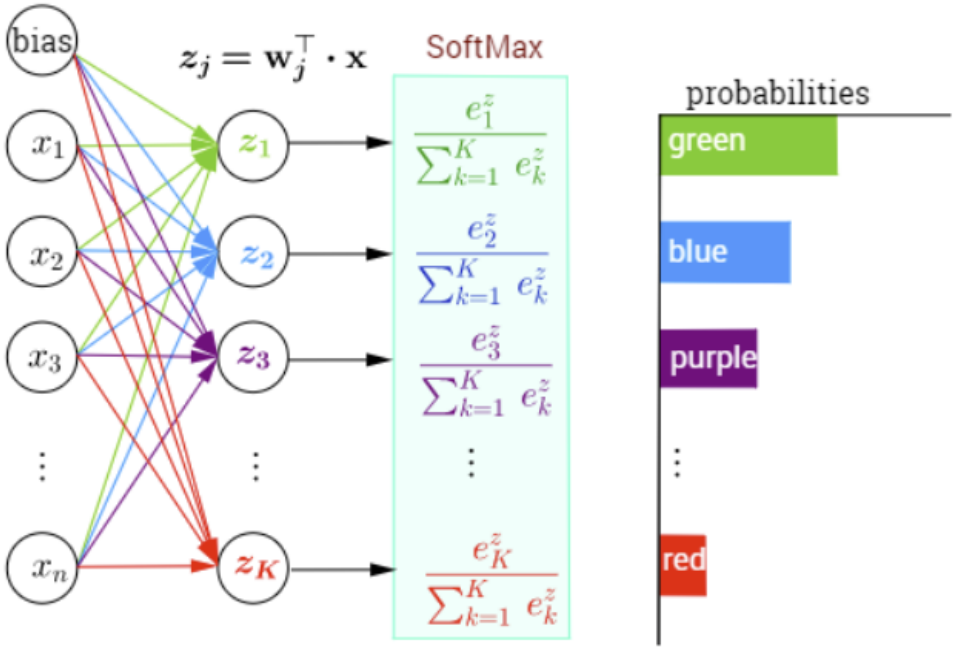
CNN Architecture

- **Other Examples**
  - **Conv-ReLU-Pool-Conv-ReLU-Pool-Conv-ReLU-Pool-FC-SM**
  - **Conv-Pool-Conv-Pool-Conv-FC-FC-SM**
  - **Conv-Pool-Conv**
  - **Conv-ReLU-Poo**

SoftMax Function

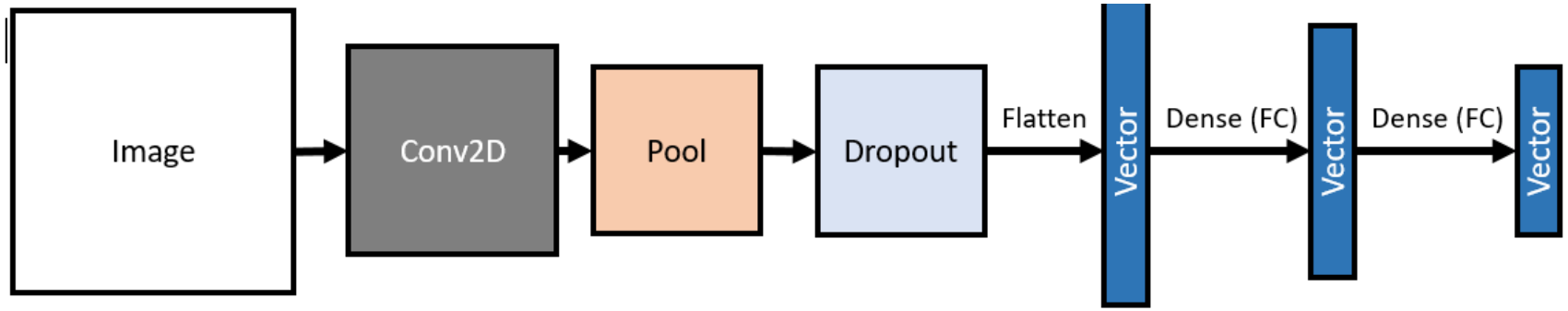
$$\sigma(j) = \frac{\exp(\mathbf{w}_j^\top \mathbf{x})}{\sum_{k=1}^{K} \exp(\mathbf{w}_k^\top \mathbf{x})} = \frac{\exp(z_j)}{\sum_{k=1}^{K} \exp(z_k)}$$

C-SM



CNN Architecture

- **Other**




```
In [25]: from keras import Model
         from keras.layers import Input, Conv2D, MaxPooling2D, Dropout, Flatten, Dense

         img = Input(shape=(28, 28, 1), name='input_layer')

         x = Conv2D(filters=64, kernel_size=(3, 3), activation='relu', name='conv')(img)
         x = MaxPooling2D(pool_size=(2, 2), name='maxpool')(x)
         x = Dropout(0.5, name='drop')(x)

         x = Flatten(name='flatten')(x)

         x = Dense(units=128, activation='relu', name='fc')(x)
         class_label = Dense(units=10, activation='softmax', name='softmax')(x)

         model = Model(inputs=img, outputs=class_label, name='simple_cnn')
```

CNN Architecture

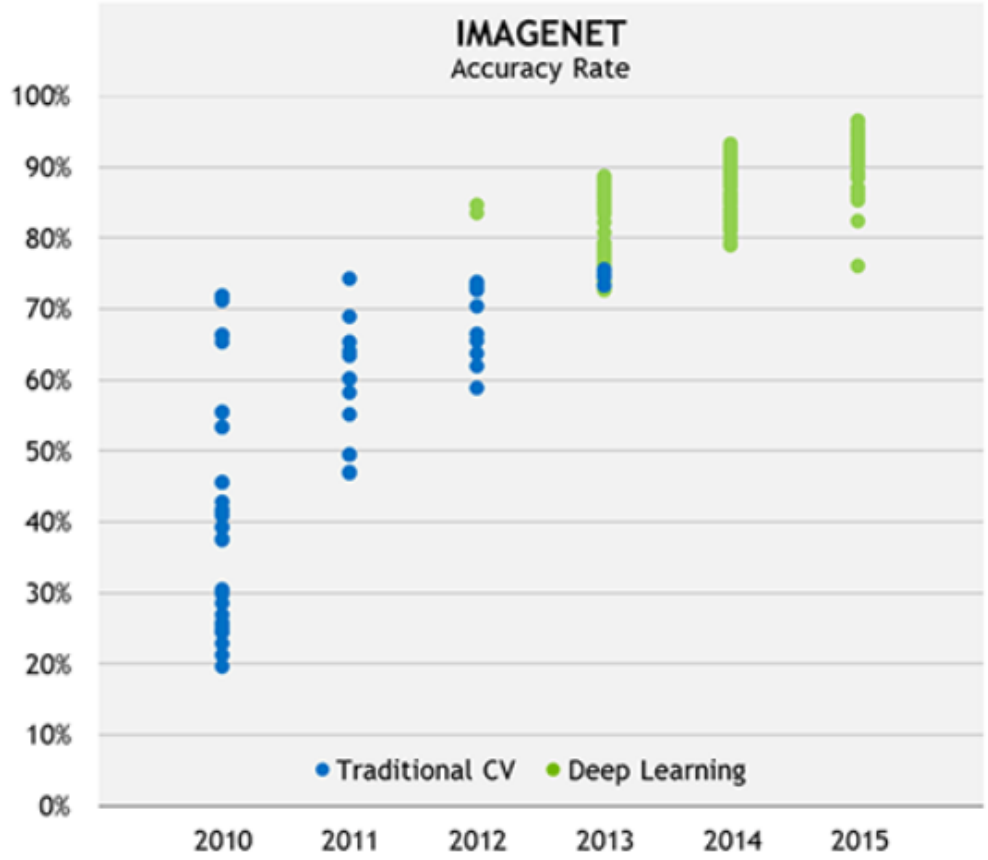
- **Example CNN (AlexNet, 2012): #-weights & Memory**

Layer	Filter/Block Size	Number of Filters	Stride	Padding	Number of Nodes (Output Size)	Number of Parameters
Input					224 × 224 × 3 (= 150,528)	
Conv:1	11 × 11 × 3	96	4	3	55 × 55 × 96 (= 290,400)	(11 × 11 × 3 + 1) × 96 (= 34,944)
Pool:1	3 × 3		2		27 × 27 × 96 (= 69,984)	
Conv:2	5 × 5 × 96	256	1	2	27 × 27 × 256 (= 186,624)	(5 × 5 × 96 + 1) × 256 (= 614,656)
Pool:2	3 × 3		2		13 × 13 × 256 (= 43,264)	
Conv:3	3 × 3 × 256	384	1	1	13 × 13 × 384 (= 64,896)	(3 × 3 × 256 + 1) × 384 (= 885,120)
Conv:4	3 × 3 × 384	384	1	1	13 × 13 × 384 (= 64,896)	(3 × 3 × 384 + 1) × 384 (= 1,327,488)
Conv:5	3 × 3 × 384	256	1	1	13 × 13 × 256 (= 43,264)	(3 × 3 × 384 + 1) × 256 (= 884,992)
Pool:5	3 × 3		2		6 × 6 × 256 (= 9,216)	
FC:6					4,096	6 × 6 × 256 × 4,096 (= 37,748,736)
FC:7					4,096	4,096 × 4,096 (= 16,777,216)
FC:8					1,000	4,096 × 1,000 (= 4,096,000)

- **#-weights: 58,621,952**
- **memory: 237M**

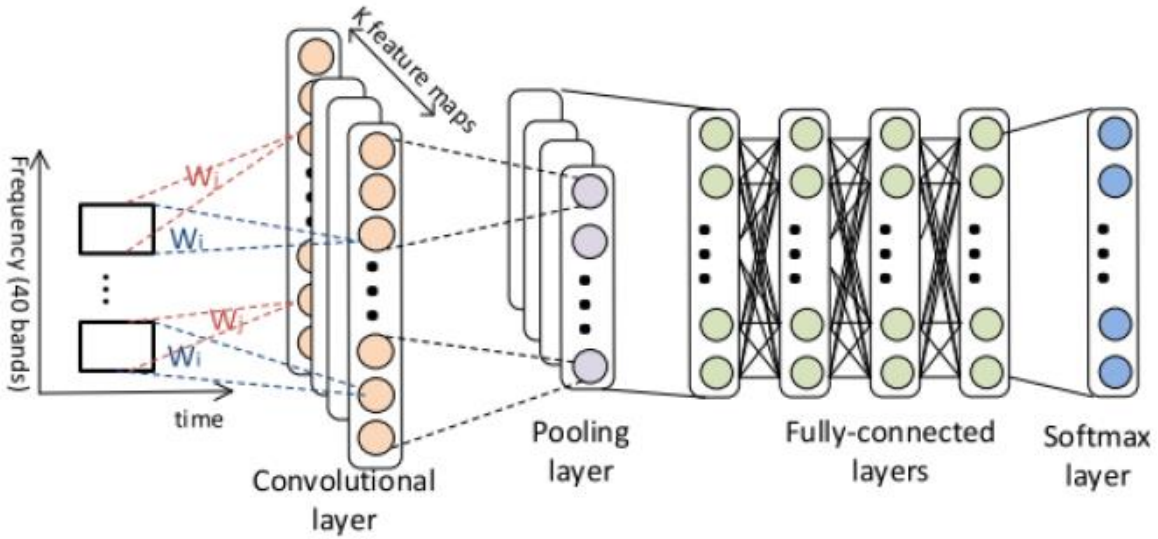
CNN - Capability

- **Breakthrough in ImageNet Challenge, since 2012**
  - AlexNet [2012]
  - GoogLeNet [2014]
  - ResNet [2015]



CNN - Capability

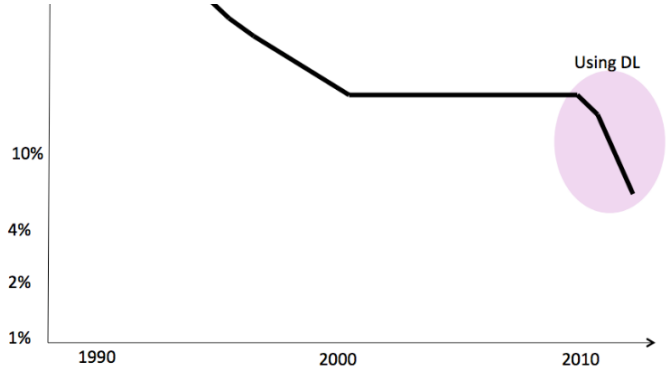
- **CNN for Speech Recognition, since 2010**



CNN - Capability

CNN for Speech Recognition

- Gaussian Mixture Model had been the standard for 30 years in the industry



- DL is more accuracy, robust to noise, and blazingly fast!
- Convolutional Neural Networks for Speech Recognition

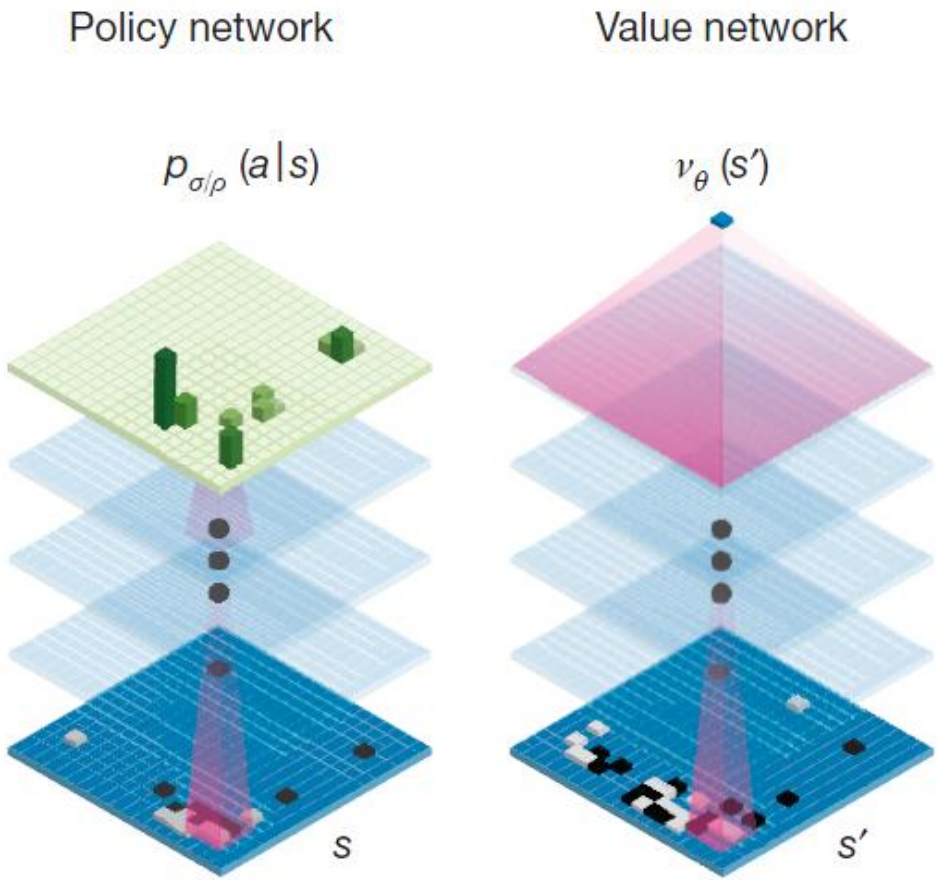
[1] Ossama Abdel-Hamid, Abdel-rahman Mohamed, Hui Jiang, Li Deng, Gerald Penn, and Dong Yu, "Convolutional Neural Networks for Speech Recognition", IEEE/ACM Transactions on Audio, Speech and Language Processing, Volume 22 Issue 10, pp. 1533-1545 (2014)

[2] Jui-Ting Huang, Jinyu Li, and Yifan Gong, "An Analysis of Convolutional Neural Networks for Speech Recognition", ICASSP (2015)

CNN - Capability

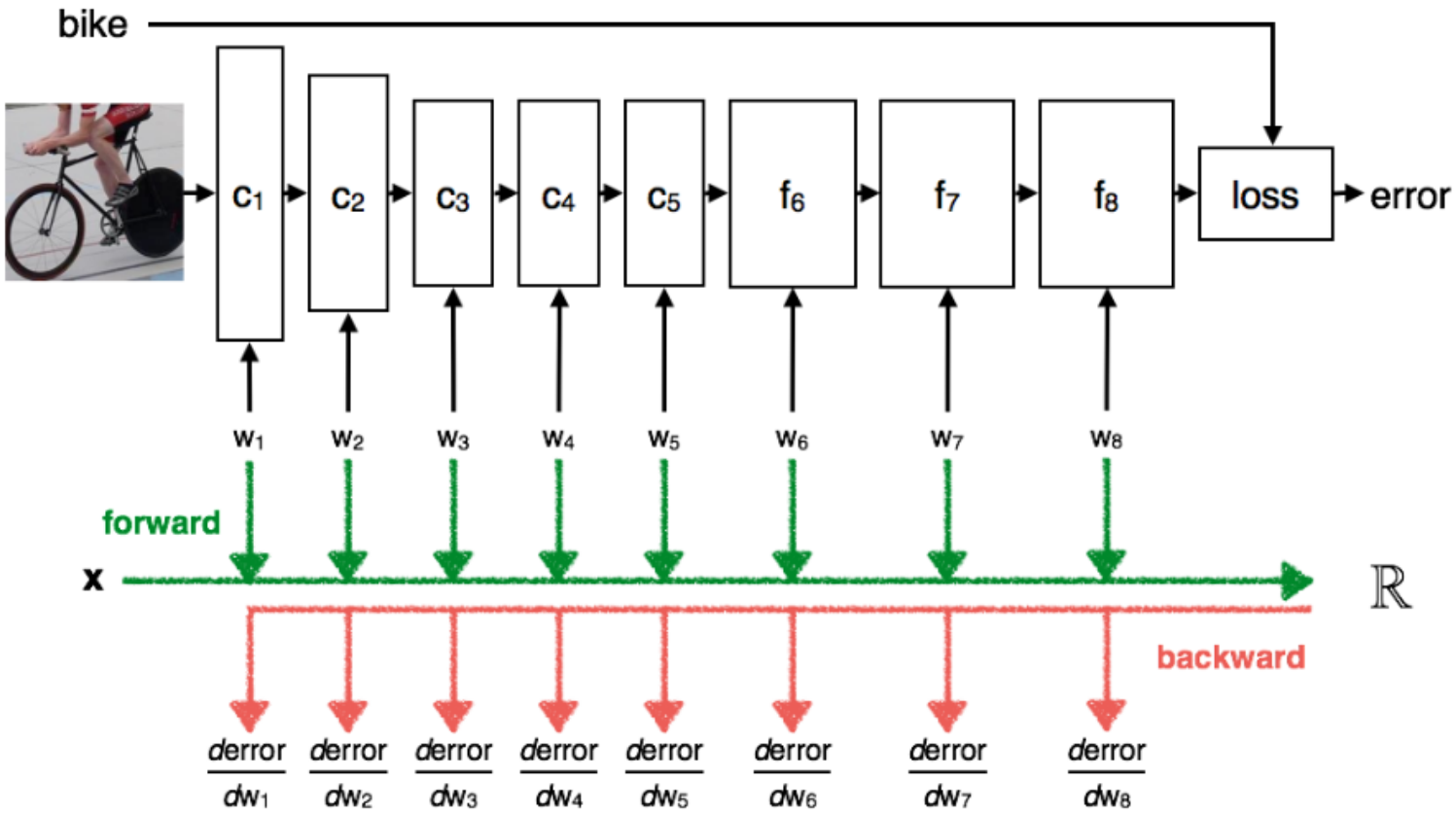
- **AlphaGo [Nature, 2016]**





CNN - Learning

- **Back-propagation algorithm for learning CNN**



CNN - Learning

- **FC layer: (1) forward calculation**

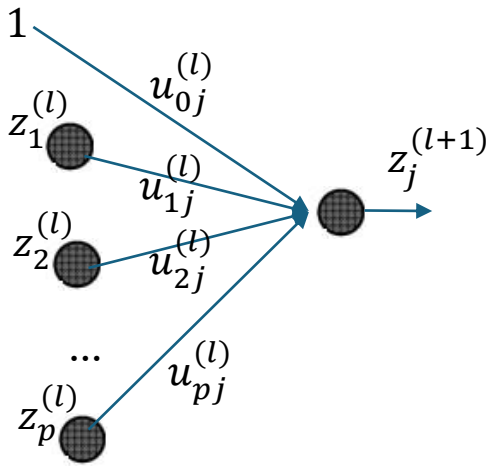
$$net_j^{(l+1)} = \sum_i z_i^{(l)} u_{ij}^{(l)}$$

$$z_j^{(l+1)} = \tau\left(net_j^{(l+1)}\right)$$

...

$$net_j^{(L)} = \sum_i z_i^{(L-1)} u_{ij}^{(L-1)}$$

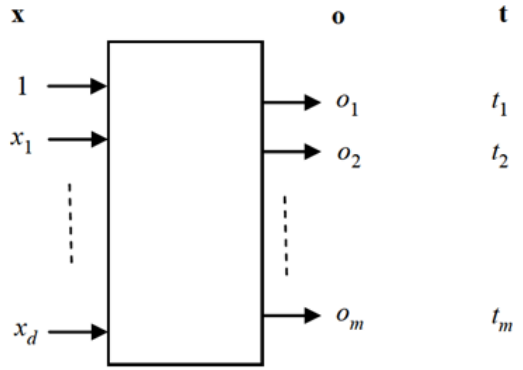
$$o_j = z_j^{(L)} = \tau\left(net_j^{(L)}\right)$$



CNN - Learning

- **FC layer: (2) error propagation**

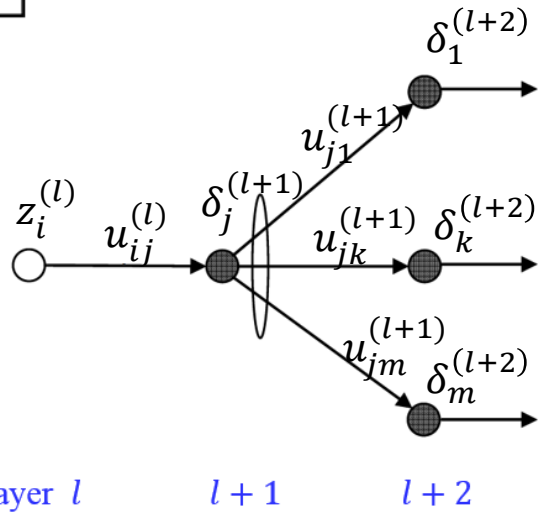
$$E = \frac{1}{2}\sum_{k=1}^{m}(o_k - t_k)^2$$



$$\delta_j^{(L)} = \tau'\left(net_j^{(L)}\right)(o_j - t_j)$$

$$\delta_j^{(l+1)} = \tau'\left(net_j^{(l+1)}\right)\sum_k u_{jk}^{(l+1)}\delta_k^{(l+2)}$$

$$l = L-1, L-2, \ldots, 1$$

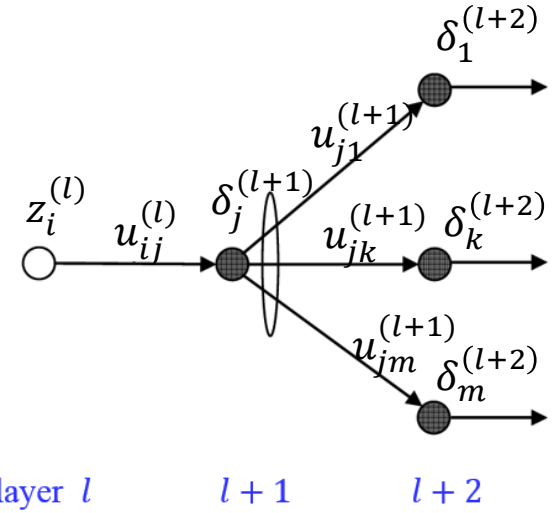


CNN - Learning

- **FC layer: (3) weight updates**

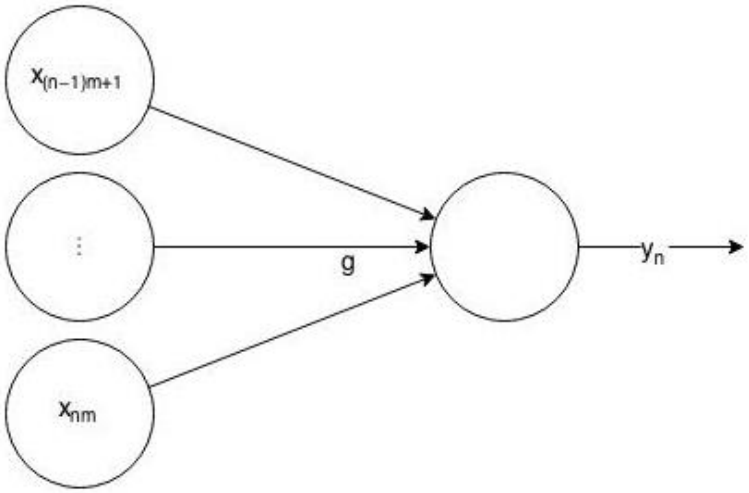
$$u_{ij}^{(l)} = u_{ij}^{(l)} - \rho \frac{\partial E}{\partial u_{ij}^{(l)}}, \qquad l = L-1, L-2, \ldots, 1$$

$$\frac{\partial E}{\partial u_{ij}^{(l)}} = \frac{\partial E}{\partial net_j^{(l+1)}} \times \frac{\partial net_j^{(l+1)}}{\partial u_{ij}^{(l)}} = \delta_j^{(l+1)} \times z_i^{(l)}$$



CNN - Learning

- **Pooling layer: (1) forward calculation**



Forward propagation

$$y = subsample(x, g)$$

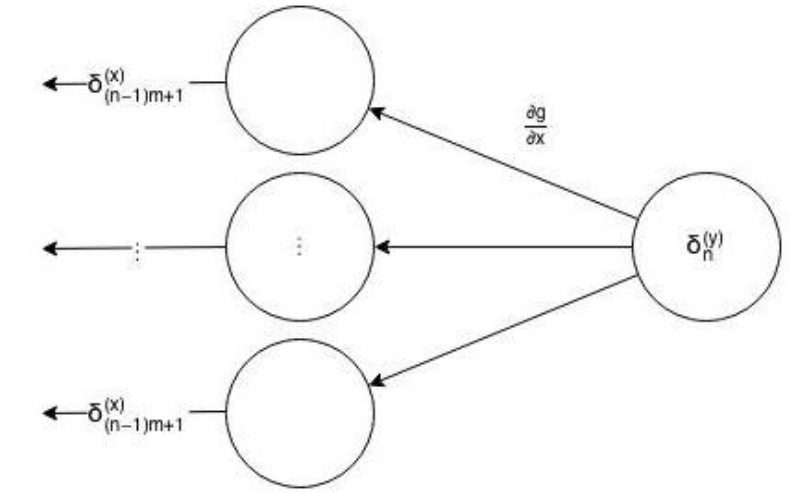
$$y_n = subsample(x, g)[n]$$

$$= g(x_1 : x_m)$$

$$g(x_1 : x_m) = \begin{cases} \dfrac{\sum_{k=1}^{m} x_k}{m} \\ \max\limits_{1 \le k \le m} (x_k) \end{cases}$$

CNN - Learning

- **Pooling layer: (2) error propagation**



Backward propagation

$$\delta^{(x)} = upsample\left(\delta^{(y)}, g'\right)$$

$$\delta^{(x)}_{1:m} = upsample\left(\delta^{(y)}_n, g'\right)$$

$$= \delta^{(y)}_n \frac{\partial g}{\partial x}$$

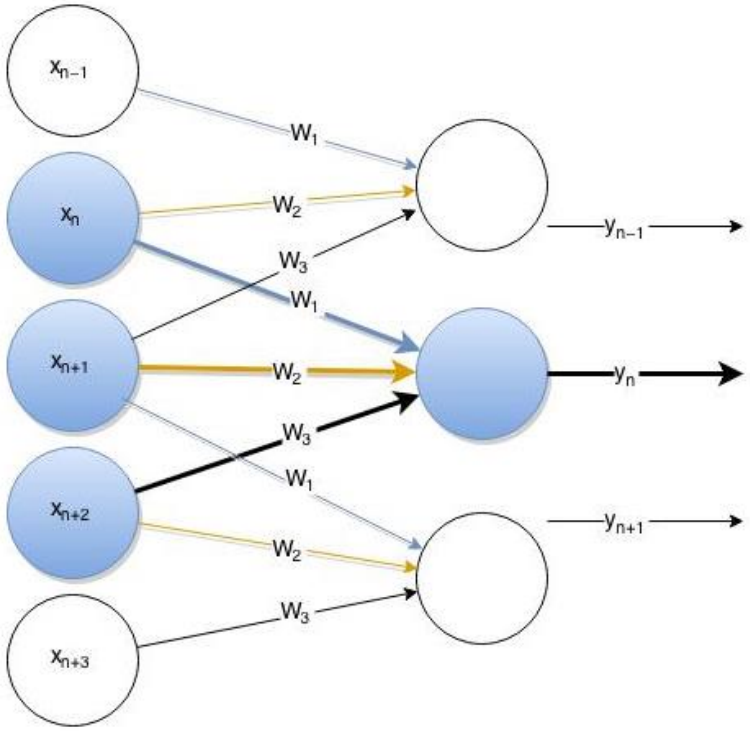
$$\frac{\partial g}{\partial x} = \begin{cases} \frac{1}{m} \\ = \begin{cases} 1, & if\,(x_i = \max(x) \\ 0, & otherwise \end{cases} \end{cases}$$

CNN - Learning

- **Pooling layer: (3) weight updates**
  - **there is no weight in pooling layer !!**

CNN - Learning

- **Convolution layer: (1) forward calculation**

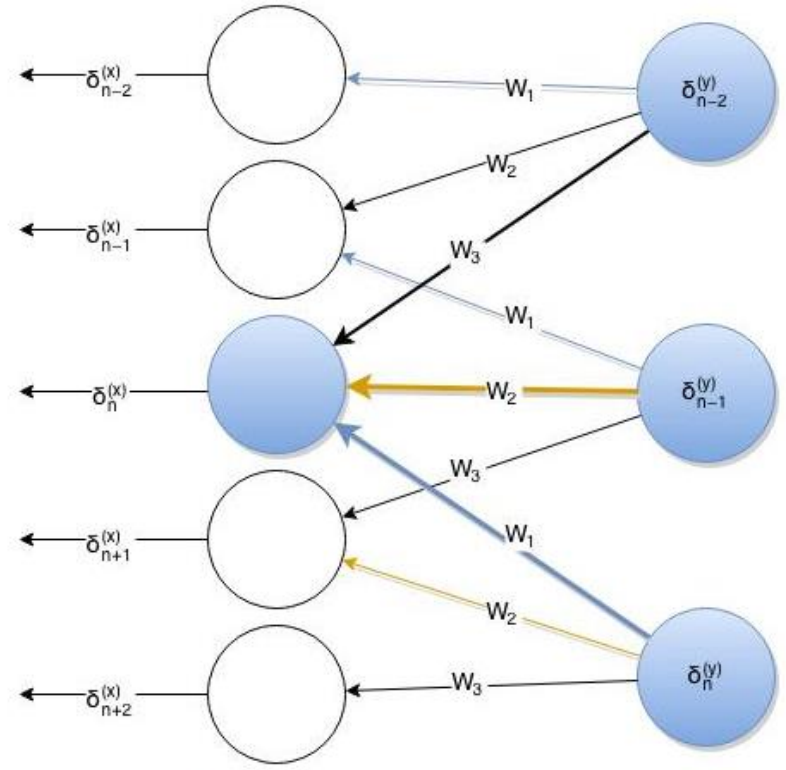


Forward propagation

$$y_n = (\mathrm{x} * \mathrm{w})[n] = \sum_{i=1}^{|\mathrm{w}|} x_{n+i-1} w_i$$

CNN - Learning

- **Convolution layer: (2) error propagation**



Back propagation for layers

$$\delta_n^{(x)} = \sum_{i=1}^{|w|} \delta_{n-i+1}^{(y)} \times w_i \times \tau'(x_n)$$

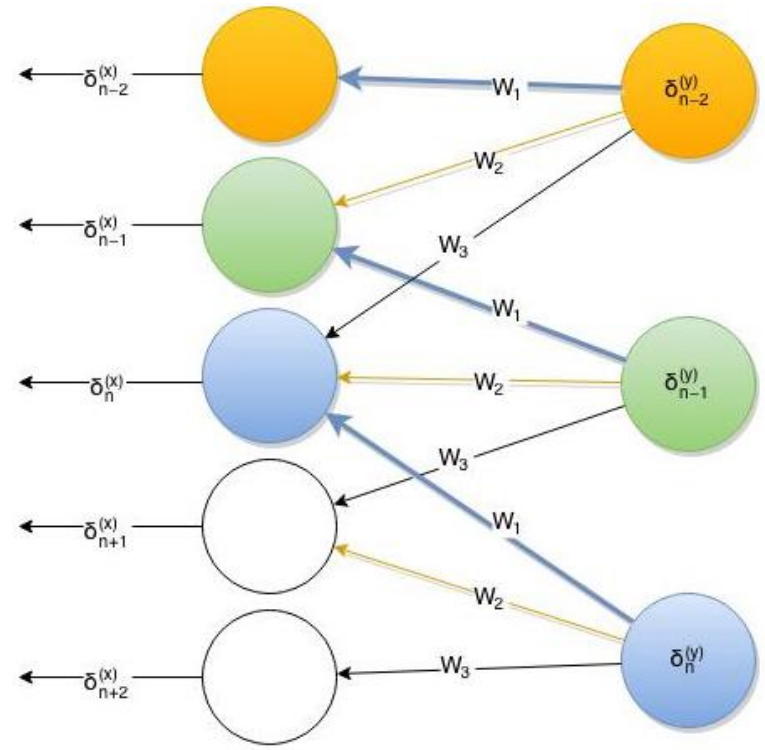
$$= \sum_{i=1}^{|w|} \delta_{n-i+1}^{(y)} \times w_i$$

$$= \left(\delta^{(y)} * flip(w)\right)[n]$$

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \xrightarrow{flip} \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$$

CNN - Learning

- **Convolution layer: (3) weight updates**



Back propagation for weights

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y} \times \tau'\big((\mathrm{x} * \mathrm{w})[i]\big) \times \frac{\partial y}{\partial w_i}$$

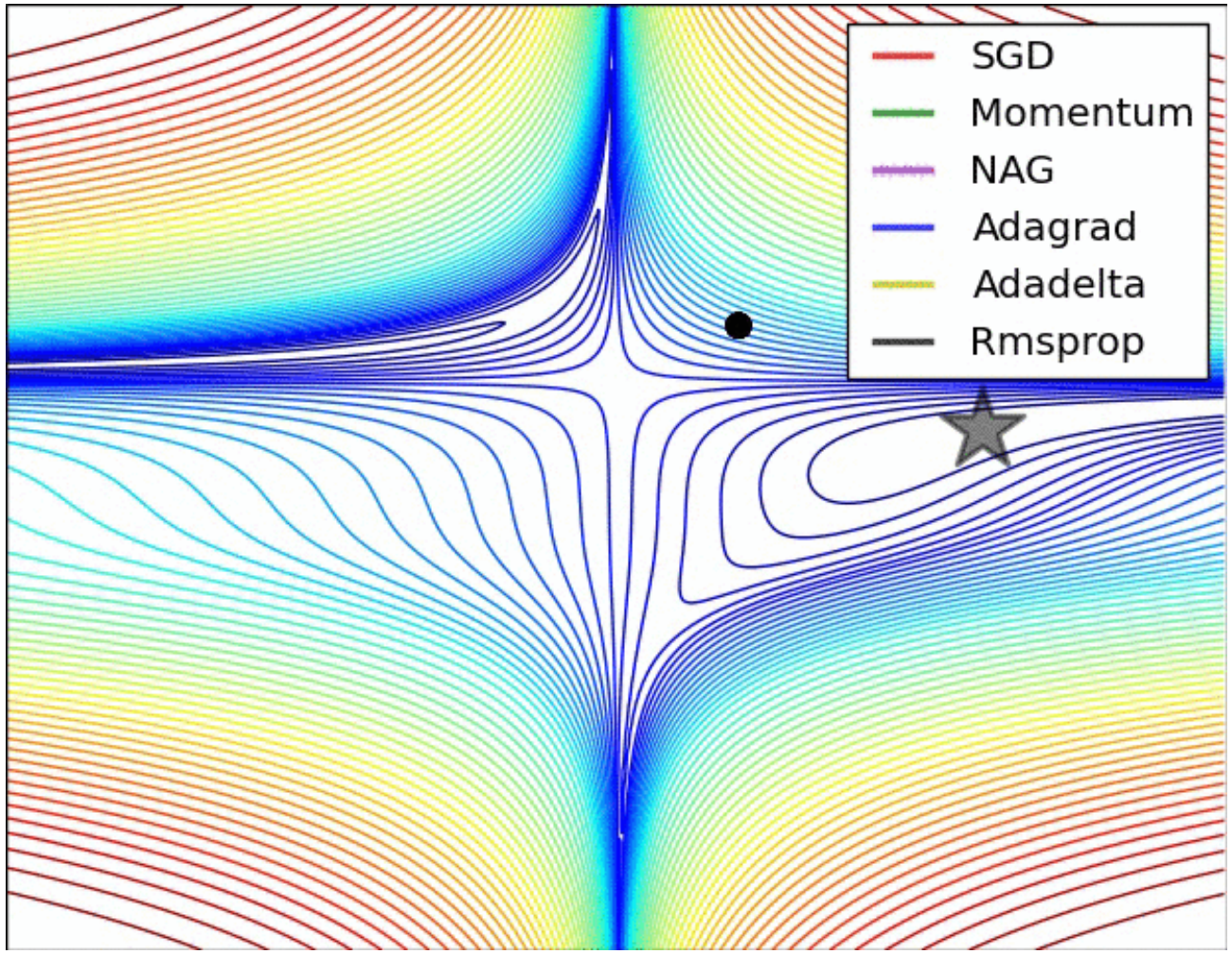
$$= \sum_{n=1}^{|\mathrm{x}|-|\mathrm{w}|+1} \frac{\partial E}{\partial y_n} \times \frac{\partial y_n}{\partial w_i}$$

$$= \sum_{n=1}^{|\mathrm{x}|-|\mathrm{w}|+1} \delta_n^{(y)} \times x_{n+i-1}$$

$$= \big(\delta^{(y)} * \mathrm{x}\big)[i]$$

$$w_i = w_i - \rho \frac{\partial E}{\partial w_i}$$

Variations of Gradient Descent



References

- Deep Learning, Y. LeCun, Y. Bengio, G. Hinton, Nature 14539, May 2015
- From Machine Learning to Deep Learning, Dongmin Kwak et al., DeepCumen, 2015
- HUB-AI Community, 2014, http://hub-ai.com/annhmmcrt/17009
- Machine Learning/Deep Learning Lecture for Everyone, Seonghun Kim, http://hunkim.github.io/ml/
- CS231n: CNN for Visual Recognition, Stanford Univ., A. Karpathy, et al., 2016, http://cs231n.github.io/convolutional-networks/
- UFLDL Tutorial, Andrew Ng, et al., http://deeplearning.stanford.edu/tutorial/
- Convolutional Neural Networks, A. Gibiansky, 2014, http://andrew.gibiansky.com/blog/machine-learning/convolutional-neural-networks/
- VGG Convolutional Neural Networks Practical, A. Vedaldi and A. Zisserman, 2015, http://www.robots.ox.ac.uk/~vgg/practicals/cnn/
- Mastering the game of Go with deep neural networks and tree search, D. Silver, et al., Nature 16961, Jan. 2016
- Analysis of CNN for speech recognition, J. Huang, et al., Microsoft, 2014

THANK YOU!